INTRODUCTION TO MACHINE LEARNING

in Python LANGUAGE

Chapter 3: Support Vector Machine

Support Vector Machine

Outline

1. SVM: Definition & Basic notions
2. Distance from a point to a line, a plane / hyperplane
3. Perception Learning Algorithm
4. Support Vector Machine

Support Vector Machine

1.1. Definition:

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression tasks. The goal of SVM is to find the optimal hyperplane in an N-dimensional space that separates data points into distinct classes. This is accomplished by maximizing the margin between the closest points from each class.

For Internal Circulation only

Support Vector Machine

1.2. Terminologies:

- **Hyperplane**: A decision boundary separating different classes in feature space, represented by the equation **ωx + b = 0** in linear classification.
- **Support Vectors**: The closest data points to the hyperplane.
- **Margin**: The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.
- **Kernel**: A function that maps data to a higher-dimensional space, enabling SVM to handle non-linearly separable data.
- **Hard Margin**: A maximum-margin hyperplane that perfectly separates the data without misclassifications.

Support Vector Machine

1.2. Terminologies:

- **Soft Margin**: Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.
- **C**: A regularization term balancing margin maximization and misclassification penalties. A higher C value enforces a stricter penalty for misclassifications.
- **Hinge Loss**: A loss function penalizing misclassified points or margin violations, combined with regularization in SVM.
- **Dual Problem**: Involves solving for Lagrange multipliers associated with support vectors, facilitating the kernel trick and efficient computation.

Support Vector Machine

2. Distance from a point to a line or a 3D plane

In 2D, the distance from a point with coordinates (x_0, y_0) to the line with the equation $\omega_1.x + \omega_2.y + b = 0$ can be determined as:

$$\frac{|\omega_1 x_0 + \omega_2 y_0 + b|}{\sqrt{\omega_1^2 + \omega_2^2}}$$

In 3D, we determine the distance from a point with coordinates (x_0, y_0) to a plane with the equation $\omega_1.x + \omega_2.y + \omega_3.z + b = 0$:

$$\frac{|\omega_1 x_0 + \omega_2 y_0 + \omega_3 z_0 + b|}{\sqrt{\omega_1^2 + \omega_2^2 + \omega_3^2}}$$

Without the absolute value, one can determine the location (the side) of the point with respect to the line (2D) or the plane (3D) !

Support Vector Machine

2. Distance from a point to a hyperplane

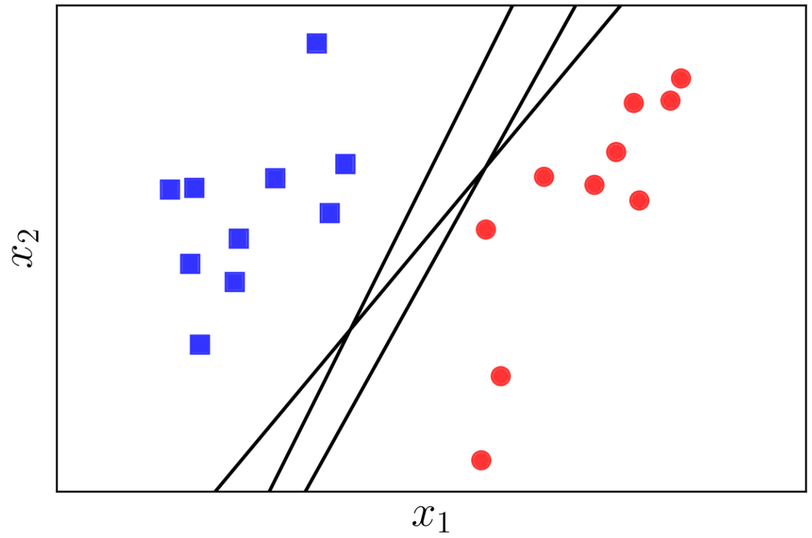
In case of n-dimension space, the distance from a point with coordinates $\mathbf{x_0} = (x_1, x_2, ..., x_n)$ to the hyperplane with the equation $\boldsymbol{\omega}.\mathbf{x} + b = 0$ can be determined as :

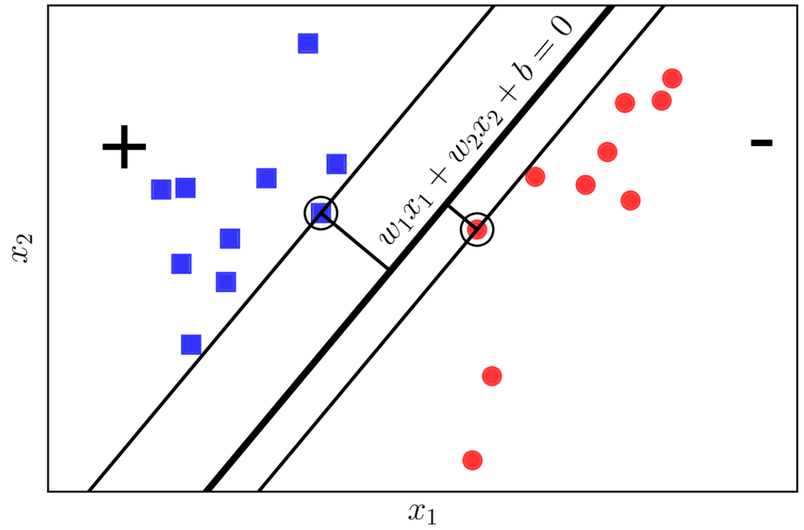
$$\frac{|\boldsymbol{\omega}^{\mathbf{T}}.\mathbf{x_0} + b|}{\sqrt{\omega_1^2 + \omega_2^2 + \ldots + \omega_n^2}}$$

Support Vector Machine

3. Perceptron Learning Algorithm

Find a hyperplane that separates 2 classes of data point: infinity of solutions → Find a criterion to find the best solution for separating the 2 classes of data points .





Support Vector Machine

4. SVM Algorithm

SVM is an advanced algorithm designed to find the optimal hyperplane (by solving a convex optimization problem) that separates classes with the maximum possible margin. It aims to identify the hyperplane that **maximizes the distance** (**margin**) between the **nearest points**, known as **support vectors**, of the two classes.

Training: SVM uses a more refined optimization technique to ensure that the margin is as large as possible, which helps the model generalize better on unseen data.

Convergence: SVM can also handle data that **isn't linearly separable** by using **kernel functions**, which map the data into higher dimensions where a linear separation is possible.

Support Vector Machine

4.1. Optimization problem of SVM

Margin: The margin is the distance between the hyperplane and the closest data points from each class. SVM aims to maximize this margin for better generalization on unseen data.

The closest data points, known as **support vectors**, are crucial in defining the decision boundary as they lie closest to the hyperplane.

To maximize the margin, one need to ensure that:

- All points from class +1 are positioned on one side of the hyperplane, while all points from class −1 are located on the opposite side.
- The margin is maximized, which means minimizing $\|\mathbf{w}\|$, the norm of the weight vector.

4.1. Optimization problem of SVM

The goal of SVM is to identify a hyperplane that divides the two classes. A hyperplane is represented by the equation:

$$\boldsymbol{\omega}^{\mathbf{T}}.\mathbf{x_0} + b = 0$$

where:

- **w** is the normal vector to the hyperplane, which defines its orientation.
- b is the bias term (which determines the offset of the hyperplane).

4.1. Optimization problem of SVM

The optimization problem can be formulated as follow:

- **Objective**: Maximize the margin, which is equivalent to minimizing $\frac{1}{2}\|\boldsymbol{\omega}\|^2$ (the factor of 1/2 simplifies the derivative).
- **Constraint:** $y_i\left(\boldsymbol{\omega}^{\mathbf{T}}.\mathbf{x} + b\right) \geq 1$

Support Vector Machine

4.1. Dual Problem

To solve this optimization problem, we apply **Lagrange multipliers** to manage the constraints, which results in the dual problem that is easier to solve:

$$L(\boldsymbol{\omega}, b, \alpha) = \frac{1}{2}\|\boldsymbol{\omega}\|^2 - \sum_{i=1}^{n} \alpha_i \left[y_i\left(\boldsymbol{\omega}^{\mathbf{T}}.\mathbf{x} + b\right) - 1\right]$$

Where $\alpha_i \geq 0$ is the Lagrange multipliers. Dual problem: Minimize L with respect to ω and b while maximizing L with respect to α

The dual form of the optimization problem:

maximize $W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} \alpha_i \alpha_j \, y_i y_j x_i^T x_j$

Under the condition that:

$\alpha_i \geq 0 \quad and \quad \sum_{i=1}^{n} \alpha_i y_i = 0$

4.1. Dual Problem

We calculate the partial derivatives of the Lagrangian with respect to ω and b, then set them equal to zero to determine the optimal values of ω and b.

$$\frac{\partial L}{\partial \omega} = 0 \Rightarrow \omega = \sum_{i=1}^{n} \alpha_i y_i x_i \; ; \qquad \frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0$$

Substituting these results into the Lagrangian, we obtain the dual form of the optimization problem:

$$maximize\ W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j\, y_i y_j x_i^T x_j$$

Under the condition that:

$$\alpha_i \geq 0 \quad and \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

4.2. Soft Kernel

When data is **non-linearly separable**, i.e. it can't be separated by a straight line or hyperplane, SVM algorithm cannot converge. In this case, **Kernel Trick** might be used. The Kernel trick enables the SVM to operate in a higher-dimensional space where the data may become separable, without directly computing the transformation. This is achieved through the use of a **kernel function**.

In Support Vector Machines (SVMs), the decision function relies on the dot products of vectors in the input space. Kernel functions replace these dot products with a non-linear function that computes the dot product in a higher-dimensional space.

Support Vector Machine

4.2. Soft Kernel

Types of **kernel function**:

- Linear Kernel: No transformation is required since the data is assumed to be linearly separable.
- Polynomial Kernel: Transforms inputs into a polynomial feature space, improving the classifier's ability to capture interactions between features.
- Radial Basis Function (RBF) Kernel: Also called the Gaussian kernel, it can capture intricate patterns by measuring the distance between points in the input space.
- Sigmoid Kernel: Replicates the behavior of neural networks by using a sigmoid function as its kernel.

The kernel trick is expressed as: $K(x, y) = \phi(x).\phi(y)$

with:

x, y are the vectors in the original input space

ϕ: is the mapping function to higher dimensional space

4.3. Final decision

Once the optimal **α** is found, we can compute the boundary parameters as follow:

$$\omega = \sum_{i=1}^{n} \alpha_i y_i x_i \; ; \quad b = y_k - \omega^T x_k$$

The classification on new data can be realized using following function:

$$f(x) = sign(\omega^T x + b)$$

Support Vector Machine

END OF CHAPTER 3